Proyecto De Software Basado En Herramientas De Integración Continua

Entrega Proyecto 1 - Escenario 3

Alejandro Montoya Sandoval

Edinson Hernández

Jonathan Franco Santana

Politécnico Grancolombiano

Integración Continua

John Olarte

10 de noviembre de 2026

Tabla de contenido

Introducción....1
Objetivo General....1
    Objetivos específicos....2
Descripción Del Proyecto....2
    Tecnologías empleadas....3
Arquitectura De La Solución....4
Comunicación Entre Los Contenedores....5
Persistencia Y Disponibilidad....5
Seguridad Y Gestión De Credenciales....6
Preparación Del Entorno De Trabajo....7
    Instalación y verificación de las herramientas....7
    Prueba inicial del motor de contenedores....9
Control De Versiones Y Repositorio....10
    Creación del repositorio local....11
    Publicación del repositorio en GitHub....12
    Incorporación de los integrantes del equipo....13
    Estructura inicial del proyecto....14
Construcción De La Infraestructura....14
    Archivo de orquestación....15
    Descarga de las imágenes....16
    Validación de la configuración....18
Despliegue Y Puesta En Marcha....18
    Creación de la infraestructura local....19
    Contenedores en ejecución....20
    Registros de arranque de los servicios....21
Verificación De La Comunicación....21
    Red Docker y direcciones asignadas....22
    Base de datos creada por la aplicación....23
    Aplicación web en funcionamiento....24
Registro De Los Cambios En El Repositorio....26
    Primer commit del proyecto....27
    Repositorio publicado en GitHub....29
Imagen Propia E Integración Continua....29
    Construcción de la imagen de la aplicación....30
    Arranque con comprobaciones de salud....30
    Prueba de comunicación con el verificador....32
    Flujo de integración continua....33
    Actualización de las ramas de trabajo....36
Conclusiones....36
Enlace A Repositorio De GitHub....37
Referencias....38

Introducción

La integración continua constituye una práctica fundamental dentro del desarrollo moderno de software, ya que permite automatizar procesos, mejorar la calidad de las aplicaciones y facilitar la colaboración entre los integrantes de un equipo de trabajo (Fowler, 2006). En ese contexto, las tecnologías de contenerización se han convertido en herramientas clave para garantizar entornos consistentes, portables y reproducibles durante todo el ciclo de vida del desarrollo.

En este documento se describe la primera entrega del proyecto GhostOps CI/CD, cuyo objetivo es implementar una solución basada en Docker con dos contenedores independientes capaces de comunicarse entre sí dentro de una misma arquitectura. El primero ejecuta la aplicación web Ghost y el segundo el sistema gestor de bases de datos MySQL.

El desarrollo se documenta siguiendo el orden real en que se llevó a cabo: la preparación del entorno de trabajo, la creación y publicación del repositorio, la construcción del archivo de orquestación, el despliegue de los servicios, la verificación de la comunicación entre ambos y, finalmente, la incorporación de una imagen propia de la aplicación y de un flujo automatizado de integración continua. Cada uno de estos pasos se acompaña de las evidencias de ejecución correspondientes.

Objetivo General

Implementar una infraestructura de software basada en contenedores Docker que permita ejecutar Ghost y MySQL como servicios independientes, demostrar la comunicación entre ambos con persistencia de datos y alojar el proyecto en un repositorio de GitHub con validación automatizada.

Objetivos específicos

- Preparar el entorno de trabajo con las herramientas necesarias y verificar su correcta instalación.
- Configurar un repositorio en GitHub para el control de versiones del proyecto y vincular a los integrantes del equipo.
- Definir la infraestructura del proyecto mediante un archivo de orquestación de Docker Compose.
- Implementar la aplicación web y el sistema gestor de bases de datos en contenedores independientes.
- Establecer la comunicación entre los contenedores mediante una red Docker definida por el usuario.
- Aplicar buenas prácticas de seguridad en la gestión de credenciales y en la exposición de puertos.
- Garantizar la persistencia de la información mediante volúmenes con nombre.
- Construir una imagen propia de la aplicación a partir de la imagen oficial de Ghost.
- Automatizar la validación del entorno mediante un flujo de trabajo de GitHub Actions.
- Documentar y evidenciar el correcto funcionamiento de la solución desarrollada.

Descripción Del Proyecto

GhostOps CI/CD es un proyecto académico que utiliza Ghost como plataforma de publicación web y MySQL como sistema gestor de bases de datos. Ghost es un gestor de contenidos de código abierto construido sobre Node.js que requiere de forma obligatoria un

motor de base de datos externo para almacenar su información, lo que lo convierte en un caso de estudio adecuado para una arquitectura de dos contenedores.

La infraestructura se implementa mediante contenedores Docker independientes, administrados con Docker Compose y conectados a través de una red privada. Esta separación responde al principio de responsabilidad única aplicado a la infraestructura: cada contenedor ejecuta un solo servicio, puede escalarse o reemplazarse de forma autónoma y declara explícitamente sus dependencias.

El proyecto se desarrolló en dos momentos. En el primero se levantó la infraestructura directamente desde las imágenes oficiales de Ghost y de MySQL, con el fin de comprobar que ambos servicios se comunicaran correctamente. En el segundo se incorporó un archivo Dockerfile propio para construir la imagen de la aplicación desde el repositorio, se fijaron las versiones exactas de las imágenes y se añadió un flujo de integración continua que valida todo el entorno en cada cambio. Ambos momentos quedan documentados en este informe.

El proyecto evolucionará hacia un pipeline completo de integración y entrega continua con Jenkins, Travis CI y Codeship en las entregas posteriores. El control de versiones se encuentra alojado en un repositorio de GitHub, en el que cada integrante trabaja sobre su propia rama y la rama principal se conserva como referencia histórica.

Tecnologías empleadas

- Ghost CMS 6, como aplicación web de publicación.
- MySQL 8.0, como sistema gestor de bases de datos relacional.
- Docker Engine y Docker Compose V2, para la contenerización y la orquestación.
- Git y GitHub, para el control de versiones del proyecto.
- GitHub Desktop y Visual Studio Code, como herramientas de trabajo del equipo.

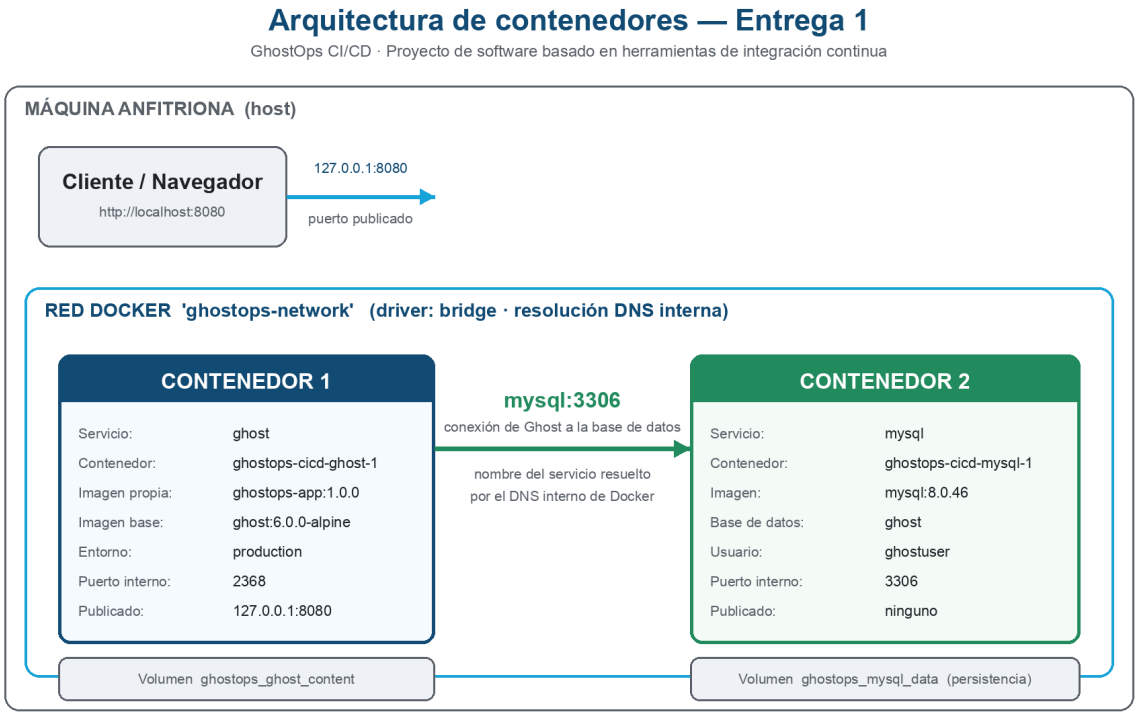
- GitHub Actions, para la validación automatizada del entorno en cada cambio.
- Windows 11 con Docker Desktop y PowerShell, como entorno de ejecución local.
- Jenkins, Travis CI y Codeship, previstos para las etapas posteriores.

Arquitectura De La Solución

La plataforma está compuesta por dos contenedores que se ejecutan de forma aislada y que se comunican exclusivamente a través de una red interna de Docker creada para el proyecto. El primer contenedor ejecuta la aplicación Ghost y es el único que publica un puerto hacia la máquina anfitriona. El segundo ejecuta el motor de base de datos y no publica ningún puerto, por lo que únicamente resulta alcanzable desde dentro de la red del proyecto.

Figura 1

Arquitectura de los dos contenedores del proyecto.



Nota. Elaboración propia a partir del archivo compose.yaml del repositorio.

Conviene señalar que la publicación del puerto de la aplicación se realiza sobre la interfaz de bucle local, mediante la declaración 127.0.0.1:8080:2368. De esta manera el servicio queda disponible únicamente desde la propia máquina y no se expone a la red exterior, lo cual constituye una buena práctica de seguridad en entornos de desarrollo.

Comunicación Entre Los Contenedores

Los servicios se conectan mediante una red Docker de tipo bridge denominada ghostops-network. Al tratarse de una red definida por el usuario, Docker habilita en ella un servidor de nombres interno que resuelve el nombre de cada servicio a la dirección que el contenedor posee dentro de la red (Docker Inc., 2026).

Por esta razón, Ghost utiliza el nombre del servicio mysql como dirección de conexión, declarado en la variable de entorno database__connection__host. No se emplea una dirección IP fija, puesto que esta cambia cada vez que el contenedor se recrea, ni tampoco localhost, ya que dentro de un contenedor ese nombre apunta al propio contenedor y no a los demás servicios de la red.

La comprobación de esta comunicación no se limita a la configuración declarada. Ghost crea su propio esquema de datos en MySQL durante el primer arranque y almacena allí cada publicación del sitio, de modo que la existencia de esas tablas y la posibilidad de consultarlas constituyen la prueba efectiva del intercambio de información entre ambos servicios.

Persistencia Y Disponibilidad

El sistema de archivos de un contenedor es efímero, de modo que la información se perdería al eliminarlo o recrearlo. Para evitarlo se definen dos volúmenes con nombre: ghostops_mysql_data, que conserva los datos de MySQL, y ghostops_ghost_content, que

conserva el contenido de Ghost. Gracias a ellos, la orden docker compose down detiene los servicios sin destruir la información del sitio.

En cuanto a la disponibilidad, MySQL incorpora una comprobación de salud mediante la directiva healthcheck, que ejecuta periódicamente la orden mysqladmin ping. Docker Compose utiliza esta comprobación junto con la condición depends_on para esperar a que la base de datos se encuentre efectivamente disponible antes de iniciar Ghost. Sin esta precaución, la aplicación intentaría conectarse antes de que el motor estuviera listo y el arranque fallaría de forma intermitente.

Seguridad Y Gestión De Credenciales

Las contraseñas de la base de datos no se escriben en el archivo de orquestación. Estas se declaran como variables de entorno obligatorias con la sintaxis de interpolación de Docker Compose, de manera que el entorno no puede levantarse si no se han definido previamente. El archivo que las contiene se encuentra excluido tanto del control de versiones como del contexto de construcción de la imagen.

- El archivo .env está ignorado por Git y por .dockerignore, de modo que las credenciales reales nunca llegan al repositorio.
- El archivo .env.example documenta las variables necesarias sin contener valores reales.
- El contenedor de MySQL no publica su puerto en la máquina anfitriona; solo es accesible desde la red interna del proyecto.
- La aplicación se publica únicamente sobre la interfaz 127.0.0.1, por lo que no queda expuesta a la red local.

- El flujo de integración continua emplea credenciales temporales de prueba, nunca las credenciales personales del equipo.

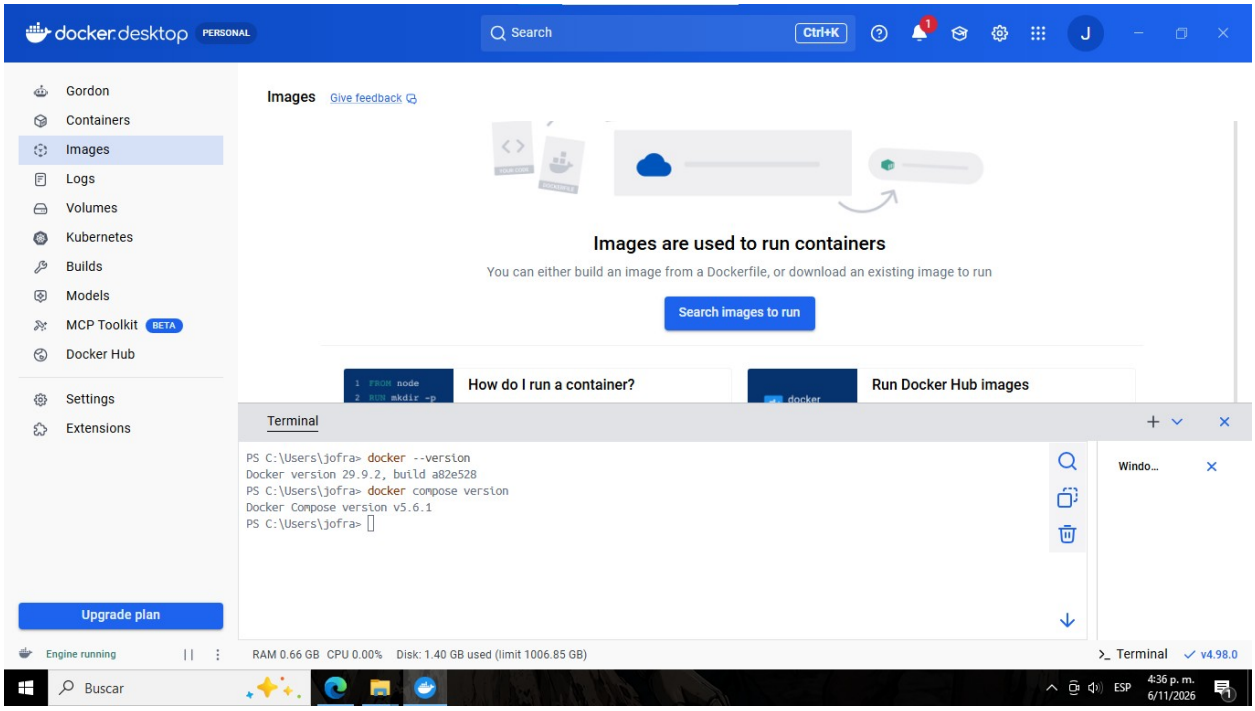
Preparación Del Entorno De Trabajo

El primer paso consistió en disponer de las herramientas necesarias en la máquina de trabajo y comprobar que estuvieran correctamente instaladas antes de iniciar la construcción del proyecto.

Instalación y verificación de las herramientas

Figura 2

Comprobación de las versiones de Docker y de Docker Compose.



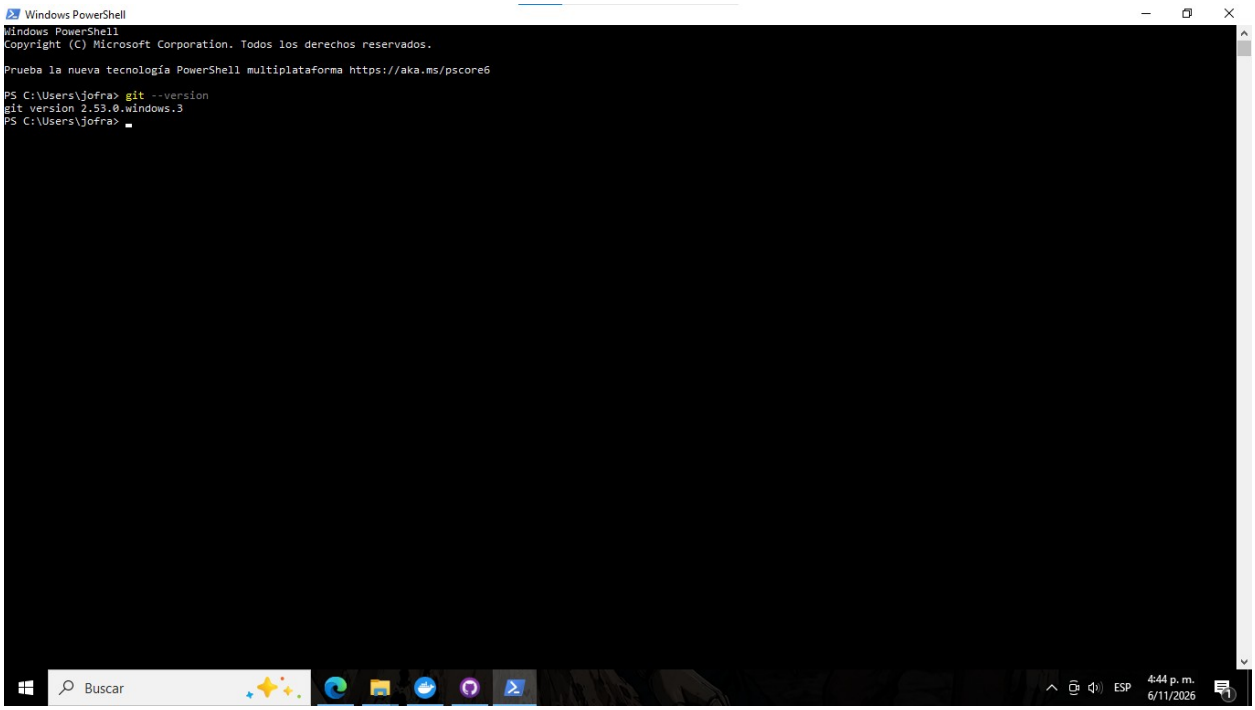
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Se verificó la instalación de Docker Desktop y se consultaron las versiones del motor y de la herramienta de orquestación desde la terminal integrada. El indicador Engine running

confirma que el servicio de Docker se encontraba activo y en condiciones de ejecutar contenedores.

Figura 3

Comprobación de la versión de Git instalada.



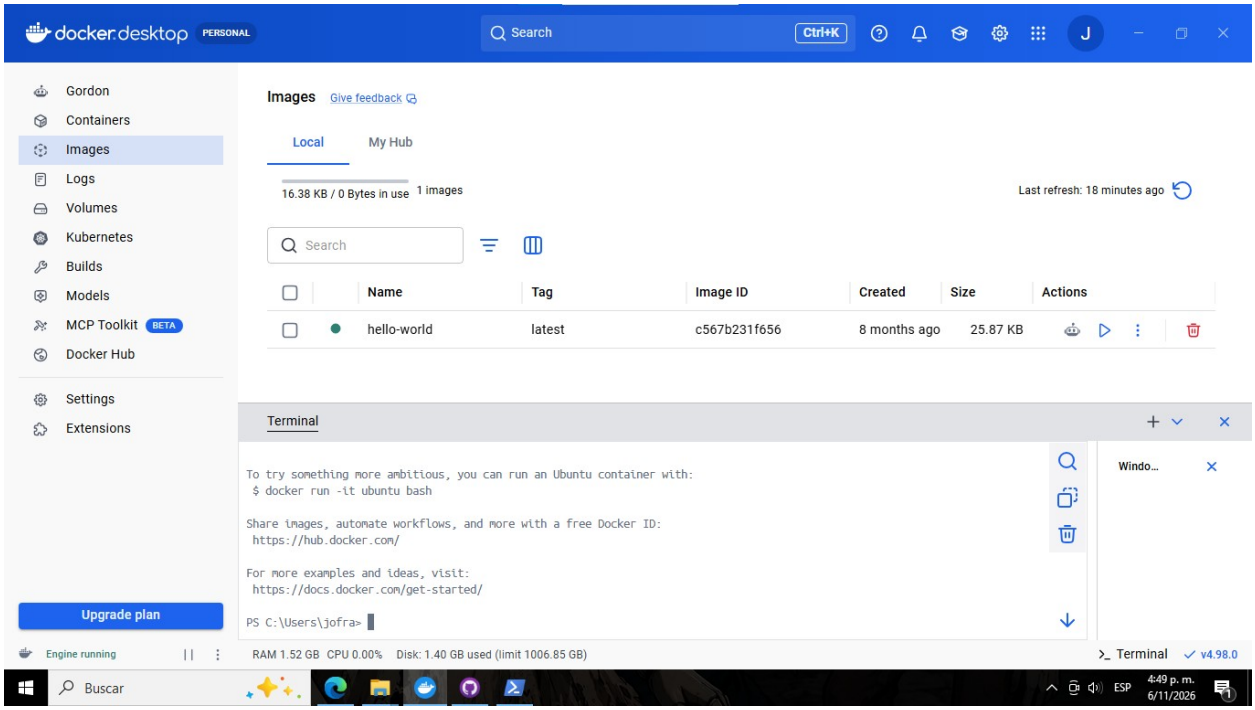
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

De igual manera se comprobó la instalación de Git, herramienta sobre la cual se apoya todo el control de versiones del proyecto y, por extensión, los procesos de integración continua que se incorporan más adelante.

Prueba inicial del motor de contenedores

Figura 4

Descarga de la imagen de prueba hello-world desde Docker Hub.

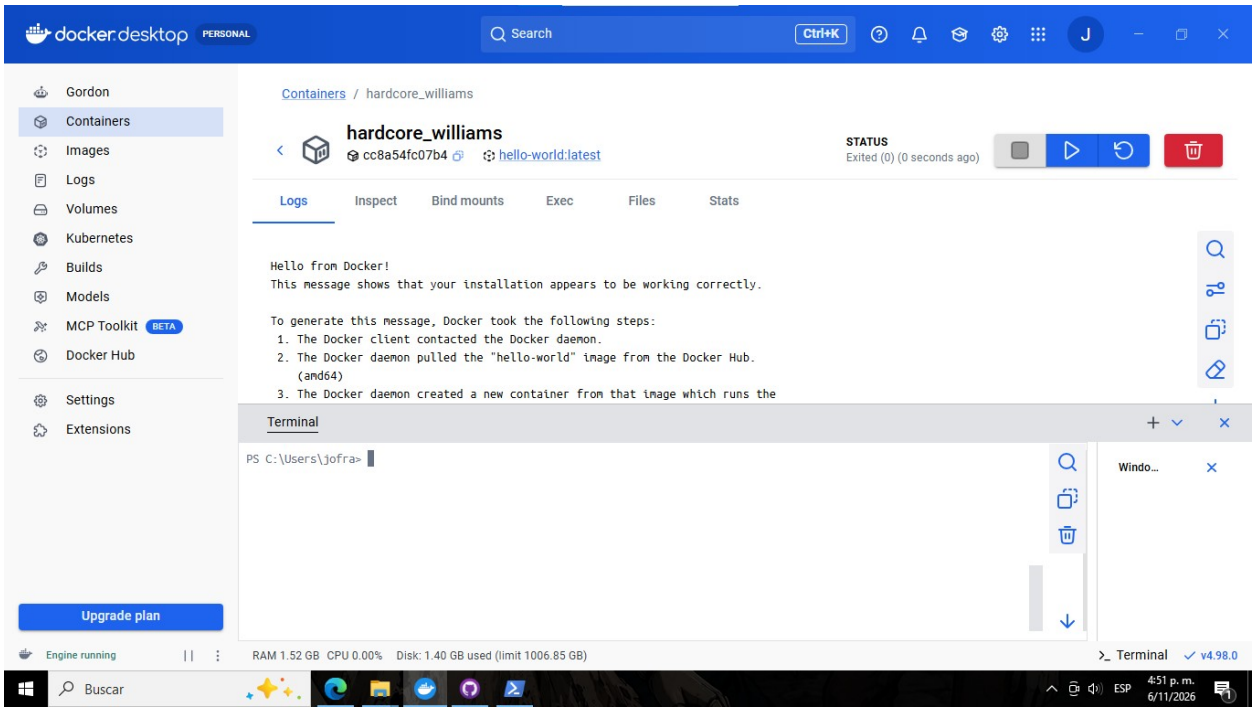


Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Antes de trabajar con las imágenes del proyecto se realizó una prueba con la imagen oficial hello-world, que permite confirmar que el equipo puede comunicarse con el registro público de imágenes y descargarlas correctamente.

Figura 5

Ejecución del contenedor de prueba y mensaje de confirmación.



Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La ejecución del contenedor devolvió el mensaje Hello from Docker!, que confirma que la instalación funciona de principio a fin: el cliente contactó al demonio, el demonio obtuvo la imagen del registro, creó un contenedor a partir de ella y ejecutó su proceso.

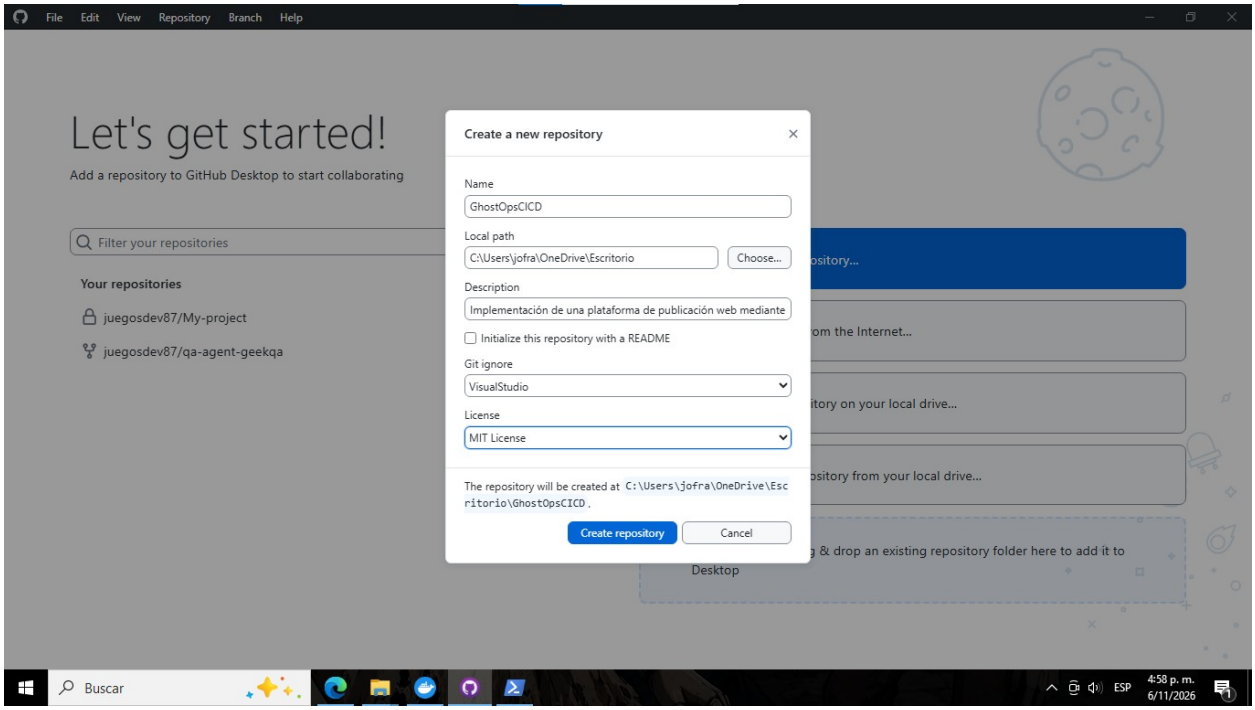
Control De Versiones Y Repositorio

Con el entorno verificado se procedió a crear el repositorio del proyecto, publicarlo en GitHub y vincular a los integrantes del equipo, de modo que todo el trabajo posterior quedara registrado bajo control de versiones desde el primer momento.

Creación del repositorio local

Figura 6

Creación del repositorio local del proyecto.



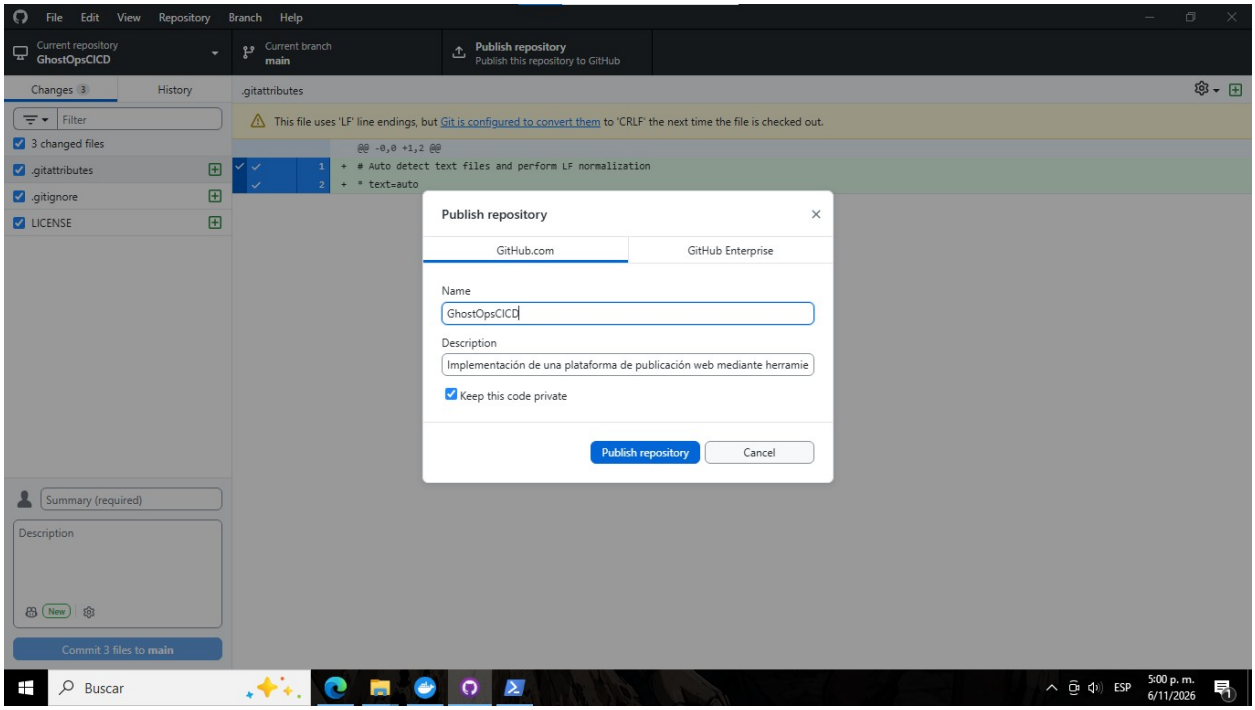
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

El repositorio se creó con el nombre GhostOpsCICD, acompañado de la descripción del proyecto, un archivo de exclusiones y la licencia MIT. Definir estos elementos desde el inicio evita que archivos temporales o credenciales lleguen por descuido al historial del proyecto.

Publicación del repositorio en GitHub

Figura 7

Publicación del repositorio en GitHub.



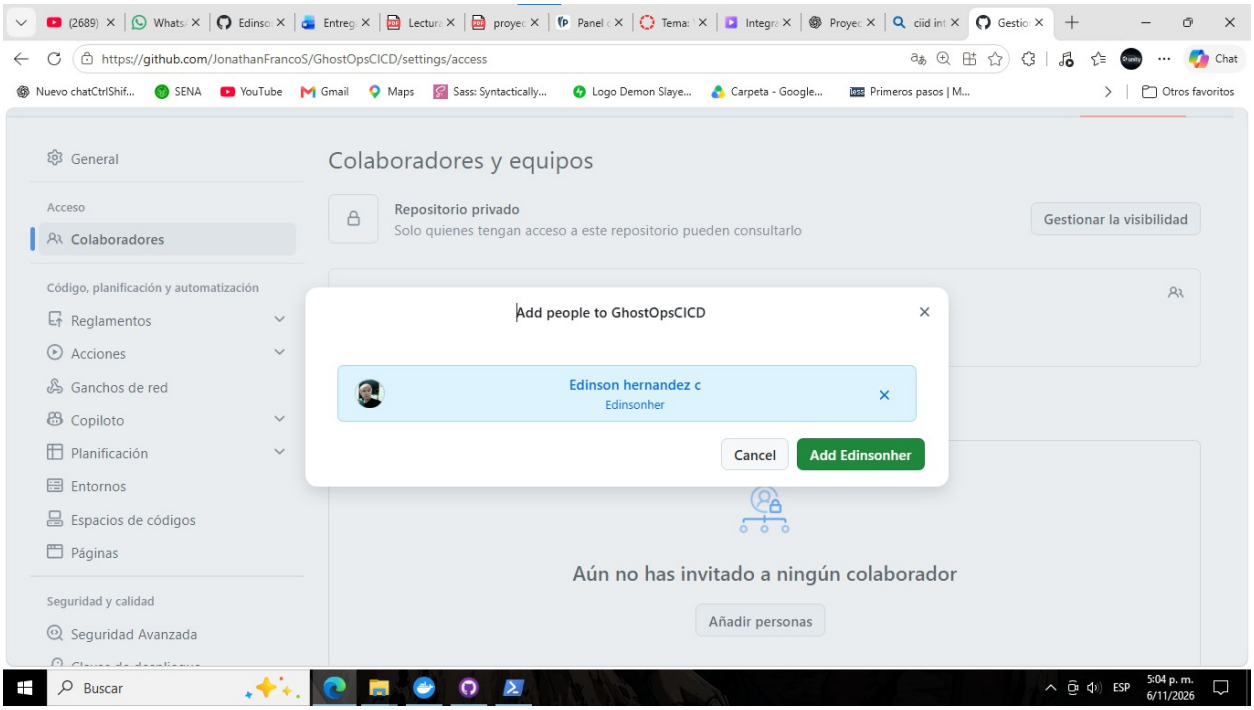
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

El repositorio local se publicó en GitHub conservando la misma descripción. A partir de este punto el proyecto dispone de un origen remoto común, requisito indispensable para el trabajo colaborativo y para la ejecución posterior de los flujos de integración continua.

Incorporación de los integrantes del equipo

Figura 8

Invitación de los integrantes del equipo como colaboradores del repositorio.



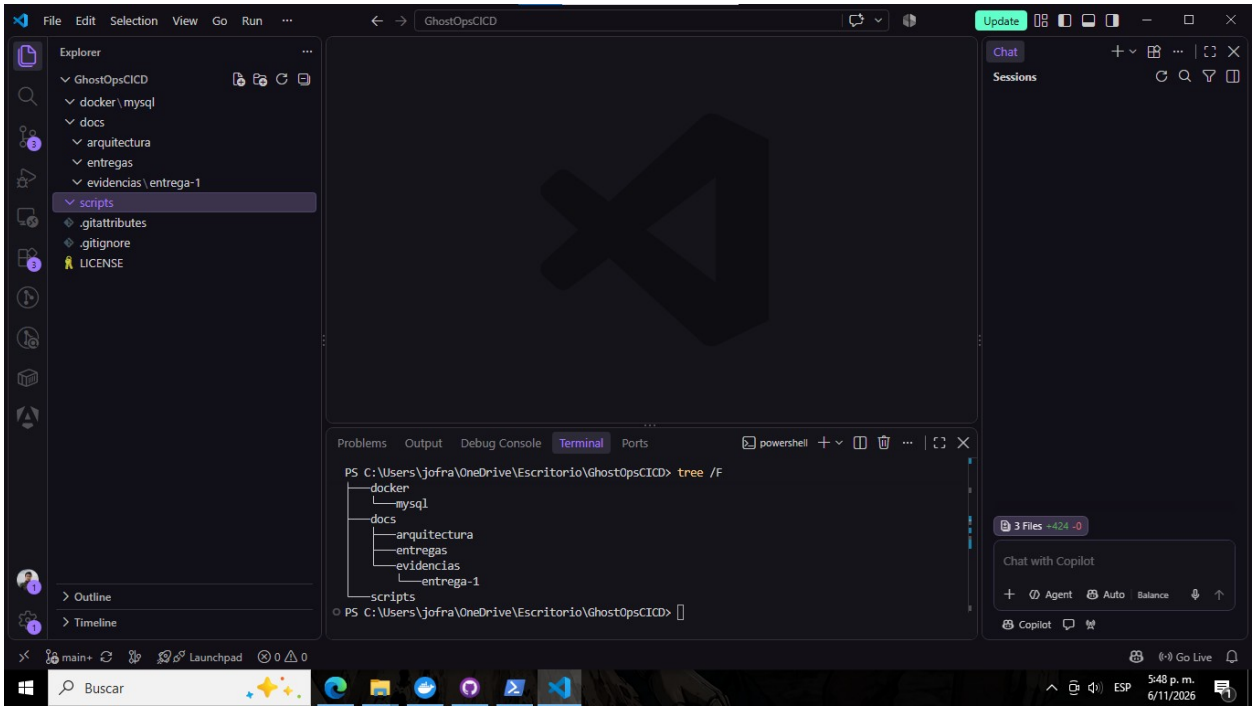
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Desde la configuración de acceso del repositorio se invitó a los demás integrantes del equipo como colaboradores. Esta operación es la que permite que cada uno trabaje sobre su propia rama y que sus aportes queden atribuidos de forma individual en el historial del proyecto.

Estructura inicial del proyecto

Figura 9

Estructura de carpetas del proyecto en el editor de código.



Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Se definió una estructura de carpetas que separa la configuración de la base de datos, la documentación, las entregas, las evidencias y los scripts auxiliares. Esta organización facilita que cada entrega del curso se incorpore sin desordenar el repositorio.

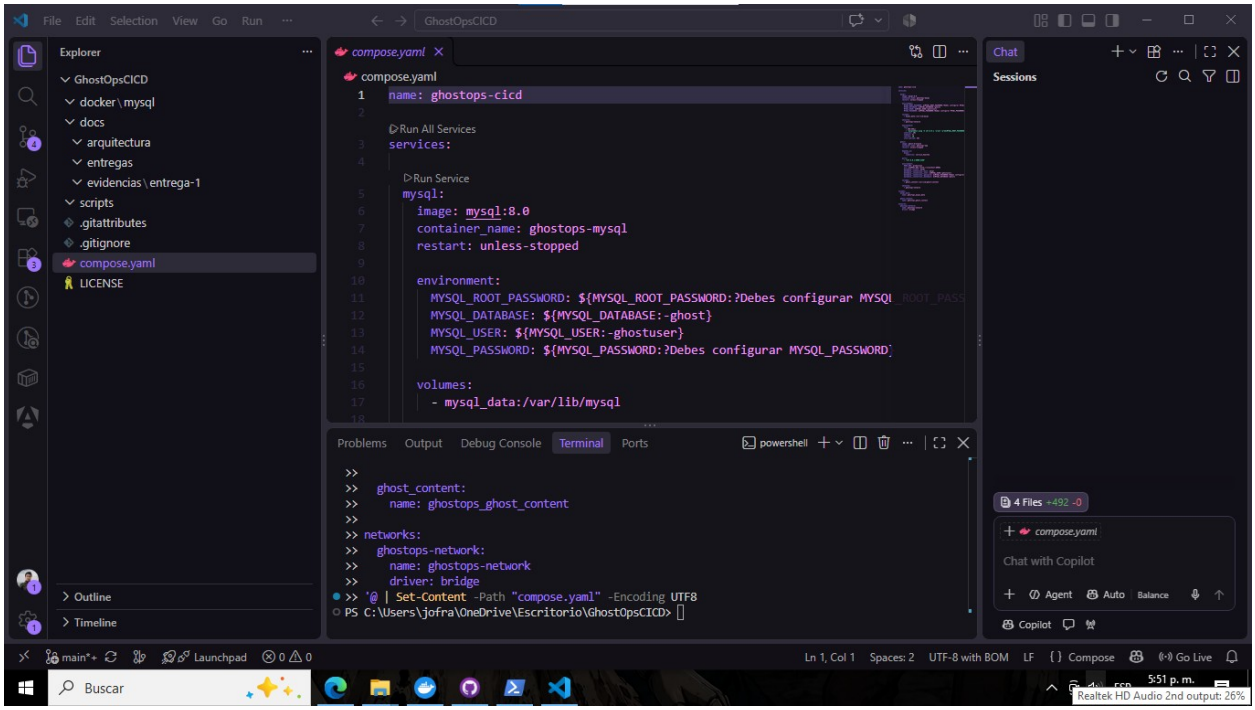
Construcción De La Infraestructura

Definida la estructura del proyecto, se procedió a declarar la infraestructura en un único archivo de orquestación y a obtener las imágenes necesarias para ejecutarla.

Archivo de orquestación

Figura 10

Archivo compose.yaml con la definición de los dos servicios.



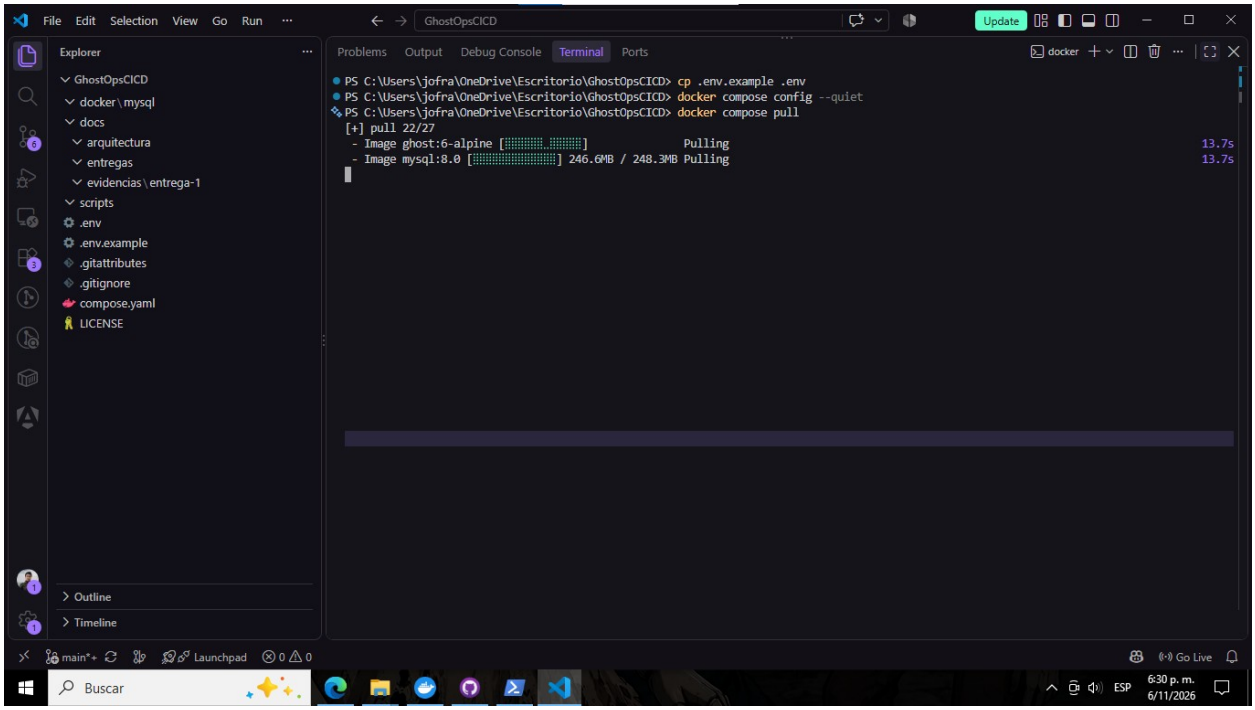
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

El archivo compose.yaml declara los dos servicios del proyecto, sus imágenes, sus variables de entorno, sus volúmenes y la red que comparten. Se aprecia que las contraseñas no figuran escritas en el archivo, sino que se declaran como variables obligatorias mediante la sintaxis de interpolación, que impide levantar el entorno si no han sido definidas.

Descarga de las imágenes

Figura 11

Preparación del archivo de variables y descarga de las imágenes.

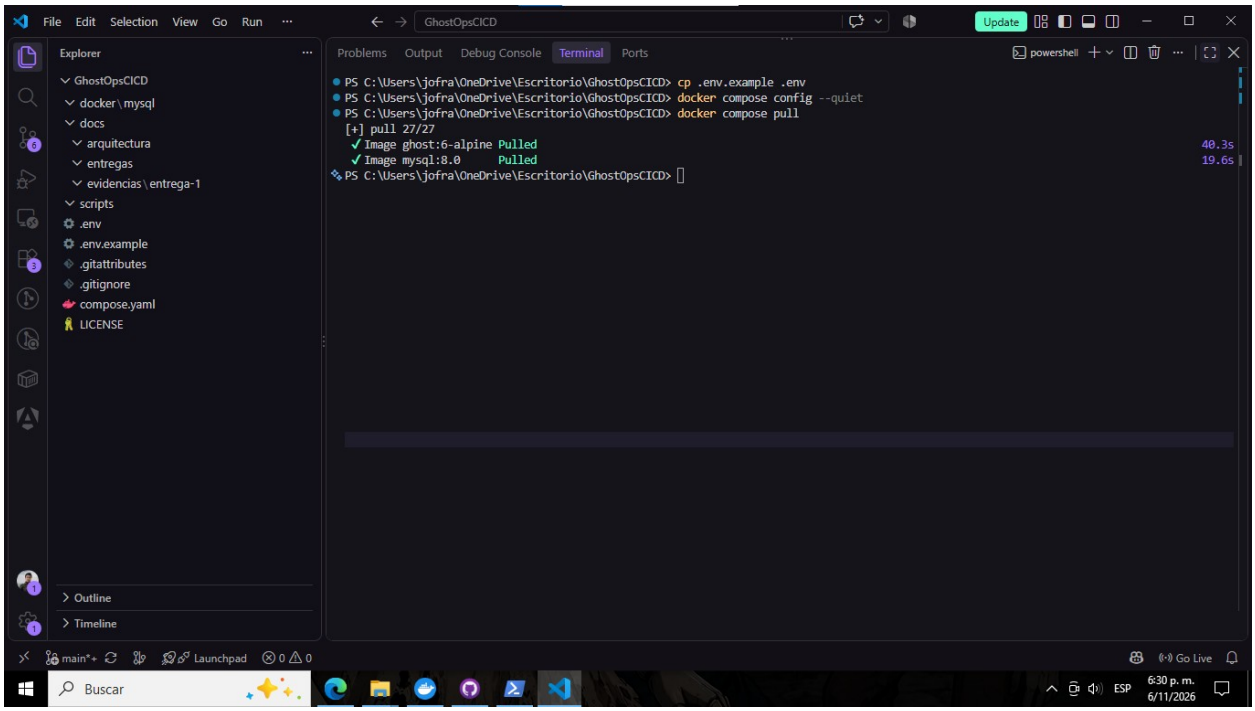


Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Se copió el archivo de ejemplo .env.example para crear el archivo .env con las credenciales locales, se validó la configuración del archivo de orquestación y se inició la descarga de las imágenes de Ghost y de MySQL desde el registro público.

Figura 12

Confirmación de la descarga completa de las imágenes.



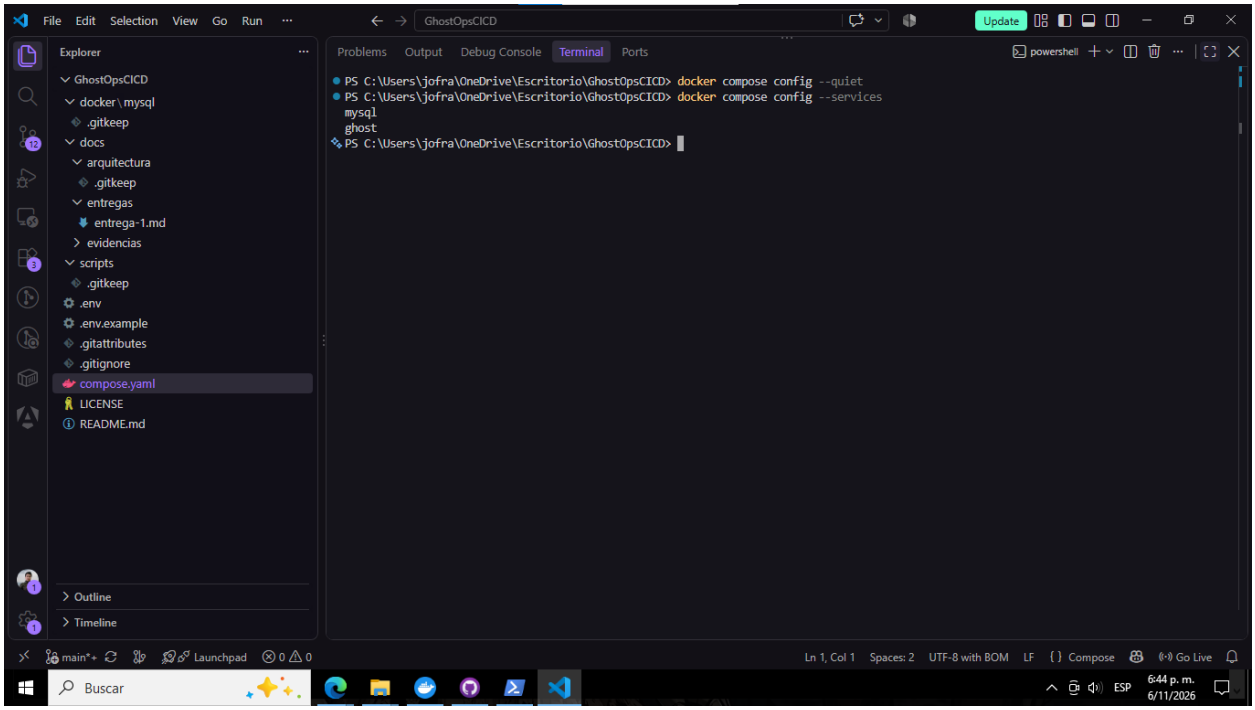
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La descarga concluyó con las dos imágenes marcadas como Pulled, lo que indica que el equipo dispone localmente de todo lo necesario para levantar la infraestructura sin depender de la red durante el arranque.

Validación de la configuración

Figura 13

Validación del archivo de orquestación y listado de los servicios declarados.



Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La orden docker compose config permitió comprobar que el archivo no contiene errores de sintaxis, y el listado de servicios confirma que la infraestructura declarada está compuesta exactamente por los dos contenedores previstos: mysql y ghost.

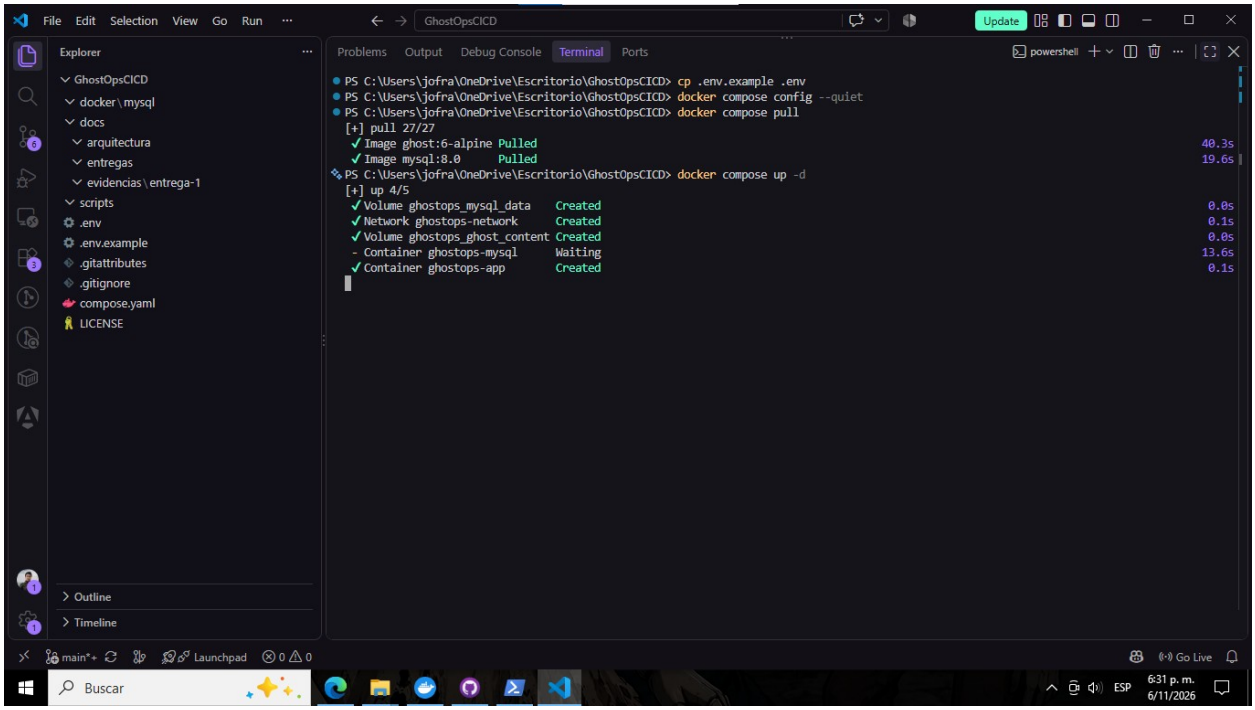
Despliegue Y Puesta En Marcha

Con la configuración validada y las imágenes disponibles se desplegó la infraestructura en segundo plano y se comprobó el estado de los servicios.

Creación de la infraestructura local

Figura 14

Creación de la red, los volúmenes y los contenedores del proyecto.



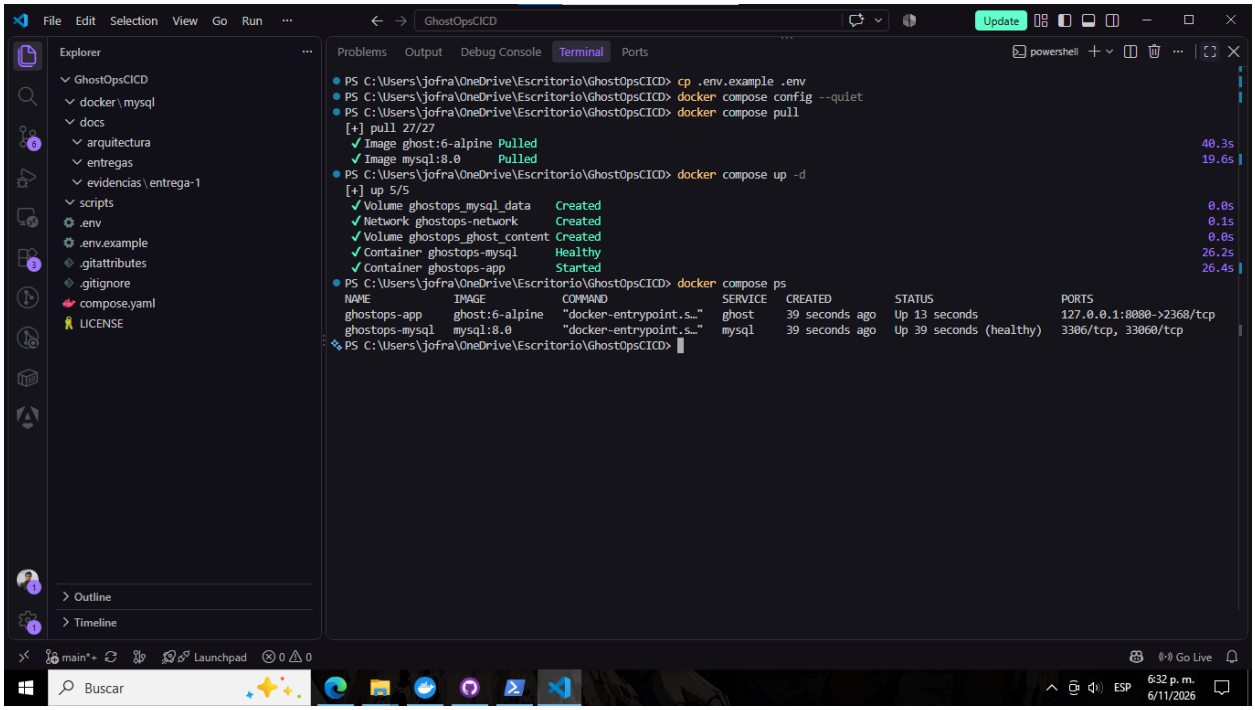
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La orden docker compose up -d creó la red ghostops-network, los volúmenes ghostops_mysql_data y ghostops_ghost_content y los dos contenedores del proyecto. Resulta especialmente significativo el estado Waiting del contenedor de la base de datos: Docker Compose no inicia la aplicación hasta que MySQL supera su comprobación de salud, tal como se había declarado en la configuración.

Contenedores en ejecución

Figura 15

Estado de los contenedores tras completarse el arranque.



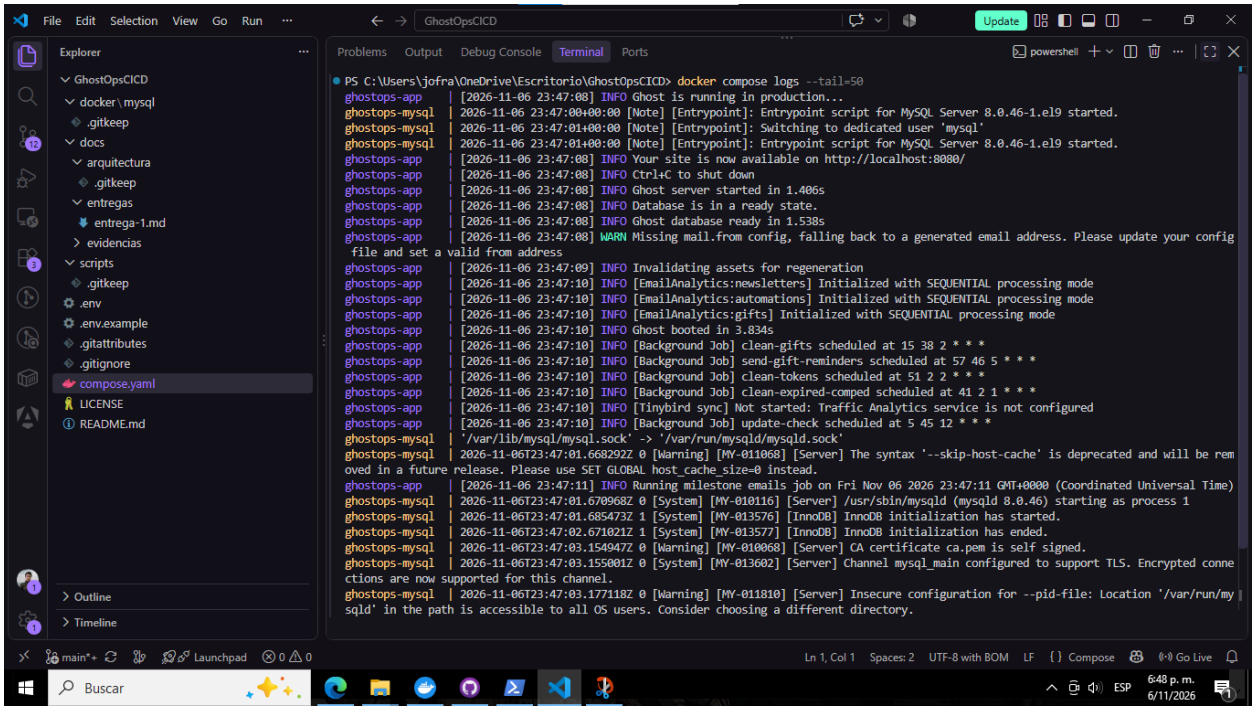
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Una vez superada la comprobación de salud, MySQL pasó al estado Healthy y la aplicación arrancó a continuación. La consulta del estado muestra los dos contenedores en ejecución y, en la columna de puertos, confirma que únicamente la aplicación publica el puerto 8080 hacia la máquina anfitriona, mientras que la base de datos mantiene sus puertos exclusivamente dentro de la red del proyecto.

Registros de arranque de los servicios

Figura 16

Registros de arranque de los dos contenedores.



Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La inspección de los registros muestra la secuencia completa de arranque. MySQL informa de la inicialización del motor en su versión 8.0.46, mientras que Ghost indica que se ejecuta en modo producción, que su base de datos está lista y que el sitio quedó disponible. El mensaje Ghost database ready confirma que la aplicación estableció la conexión con el contenedor de base de datos.

Verificación De La Comunicación

Comprobado el arranque, se verificó de forma explícita que los dos contenedores se comunicaran entre sí y que la aplicación estuviera efectivamente utilizando la base de datos.

Red Docker y direcciones asignadas

Figura 17

Inspección de la red del proyecto y direcciones asignadas a los contenedores.



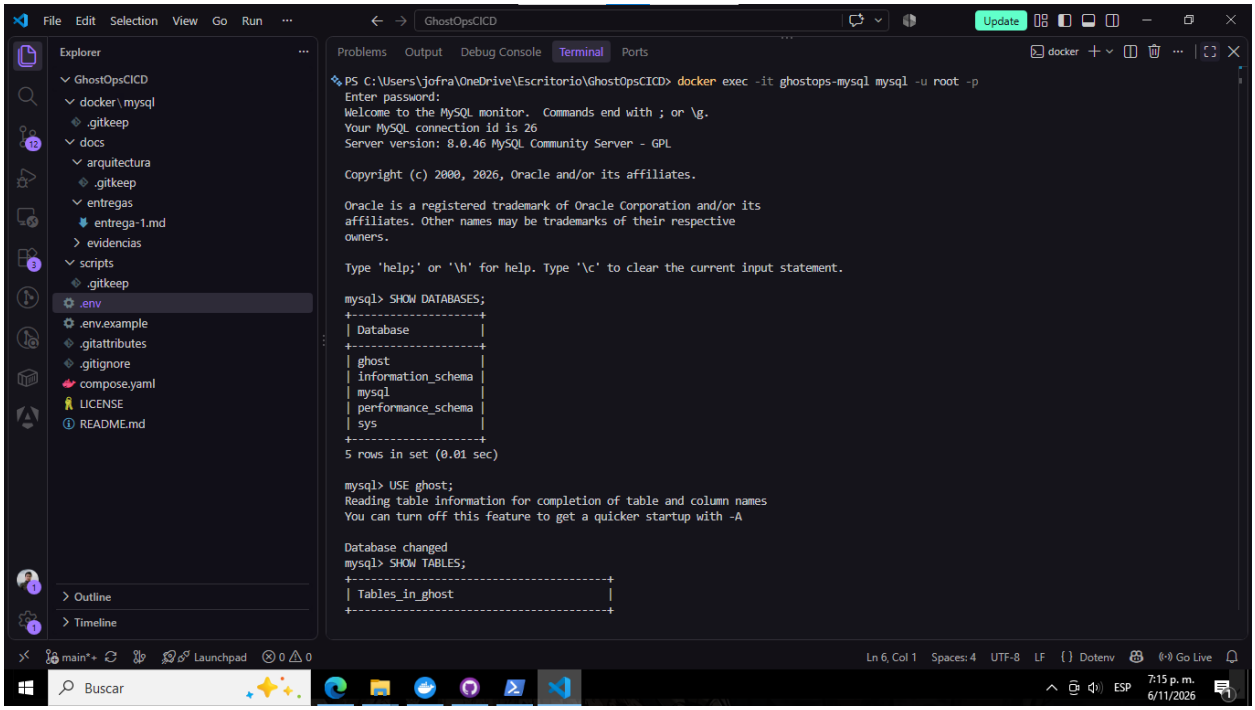
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La inspección de ghostops-network confirma que se trata de una red con controlador bridge, con la subred 172.18.0.0/16, y que los dos contenedores del proyecto están conectados a ella: ghostops-app con la dirección 172.18.0.3 y ghostops-mysql con la 172.18.0.2. Compartir esta red es la condición que habilita la resolución interna de nombres, gracias a la cual la aplicación puede alcanzar a la base de datos empleando el nombre del servicio en lugar de una dirección fija.

Base de datos creada por la aplicación

Figura 18

Consulta de las bases de datos y de las tablas creadas por la aplicación.



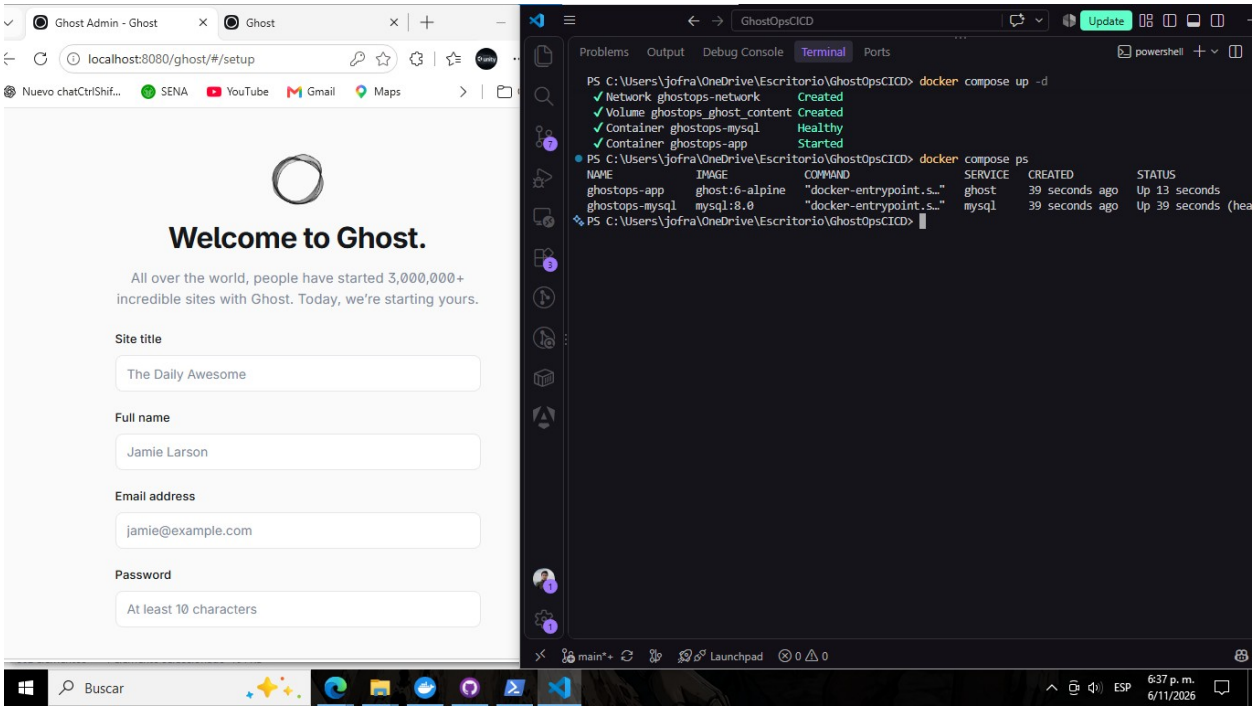
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La conexión al motor se realizó desde el interior del contenedor de MySQL. La consulta muestra la existencia de la base de datos ghost y, al seleccionarla, el conjunto de tablas que contiene. Dado que esas tablas no se crearon manualmente, sino que fueron generadas por Ghost desde el otro contenedor durante su primer arranque, esta evidencia constituye la demostración más concluyente de que el intercambio de datos entre ambos servicios se encuentra operativo.

Aplicación web en funcionamiento

Figura 19

Pantalla de configuración inicial de la aplicación junto al estado de los contenedores.

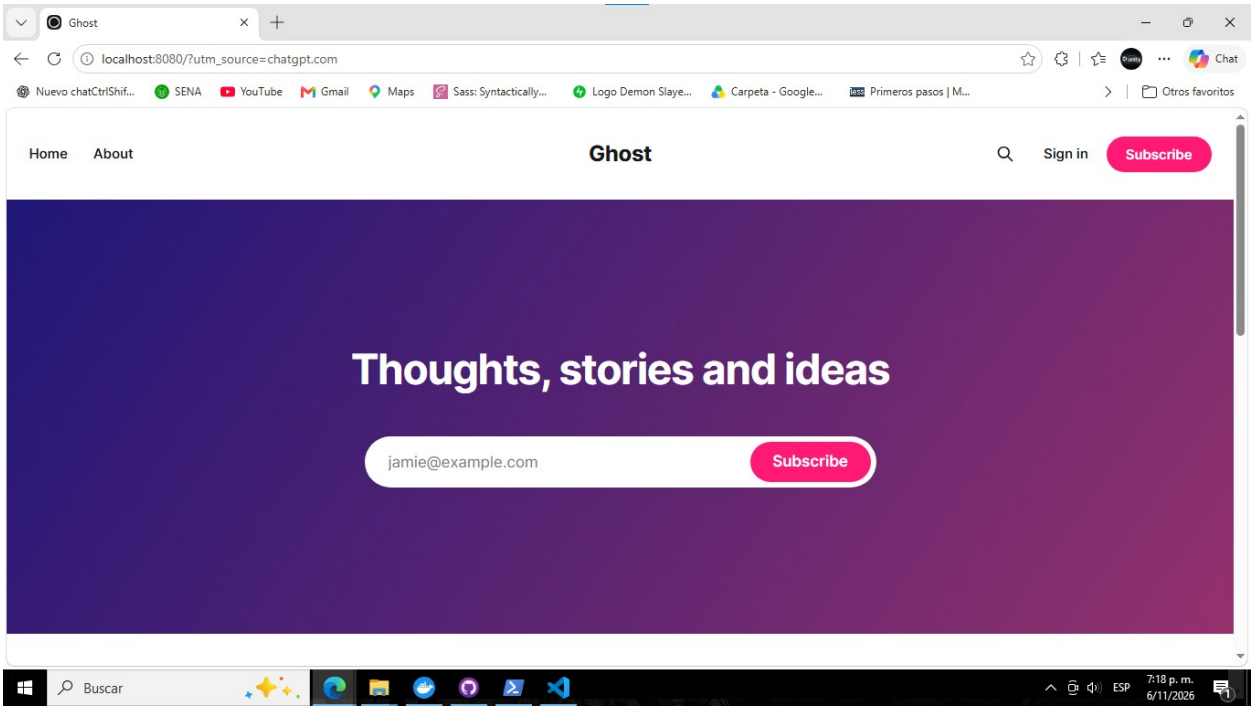


Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

La aplicación respondió en la dirección publicada con su asistente de configuración inicial. Puesto que Ghost no puede mostrar esta pantalla sin haber establecido previamente una conexión válida con su base de datos, su aparición confirma de manera indirecta, pero concluyente, la comunicación entre los dos contenedores.

Figura 20

Sitio web servido por la aplicación.

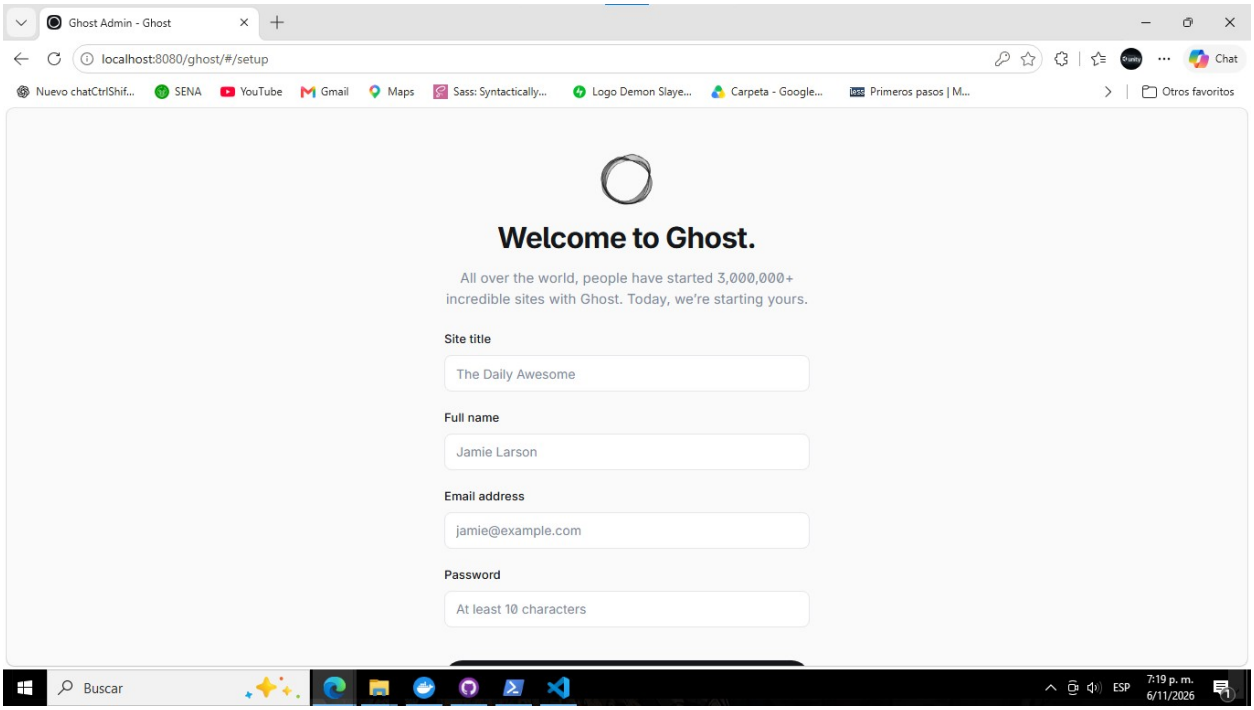


Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Completada la configuración, el sitio quedó disponible en la dirección publicada por el contenedor de la aplicación, sirviendo el contenido que lee desde la base de datos alojada en el segundo contenedor.

Figura 21

Acceso al panel de administración de la aplicación.



Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

El panel de administración, disponible en la ruta /ghost, también responde correctamente. Desde él se gestionan las publicaciones del sitio, que la aplicación escribe directamente en el contenedor de MySQL.

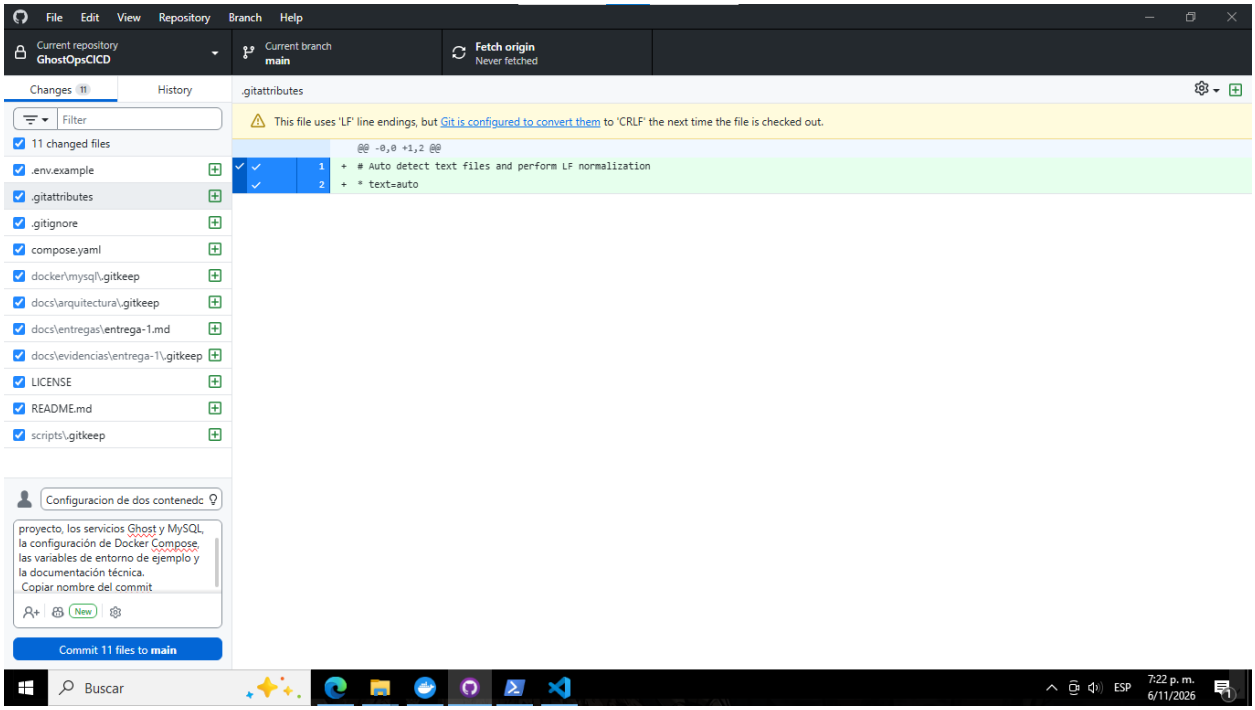
Registro De Los Cambios En El Repositorio

Verificado el funcionamiento de la infraestructura, el trabajo se incorporó al control de versiones y se publicó en el repositorio remoto del equipo.

Primer commit del proyecto

Figura 22

Preparación del primer commit con los archivos del proyecto.

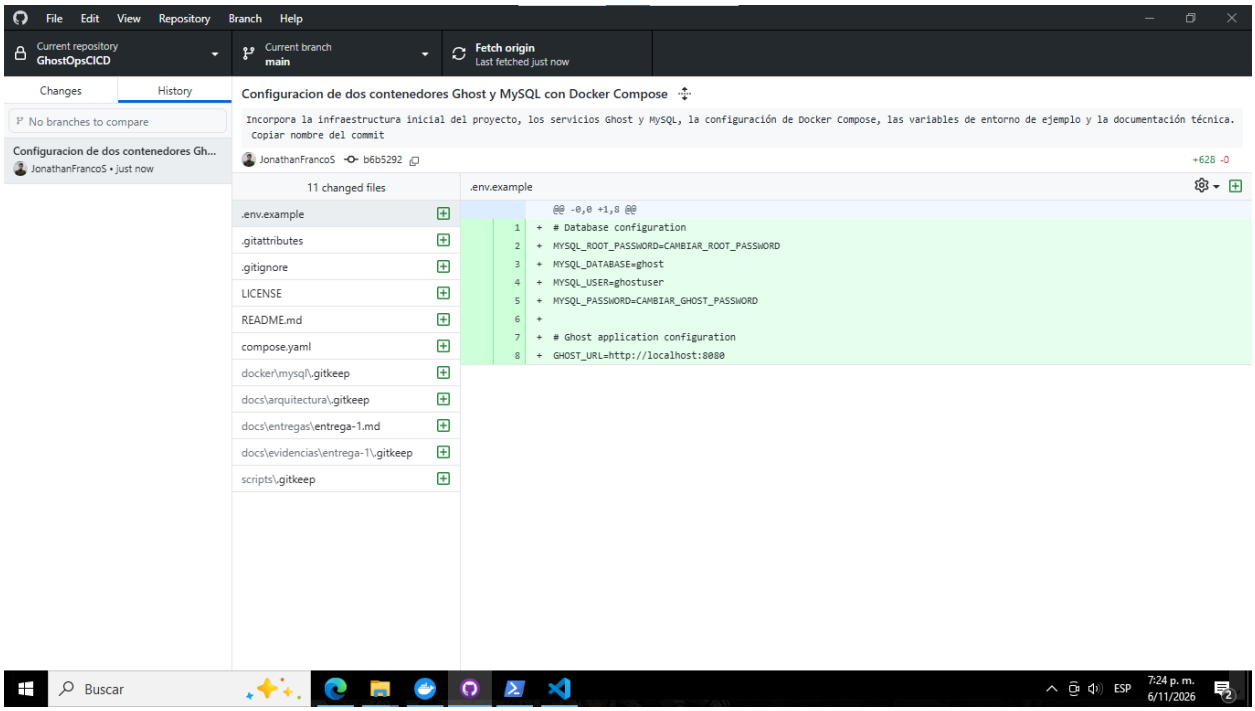


Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

Se seleccionaron los once archivos que componen la infraestructura inicial del proyecto y se redactó un mensaje de commit descriptivo. Se observa que el archivo .env no figura entre los cambios, mientras que sí lo hace .env.example, lo que confirma que las credenciales reales quedaron efectivamente excluidas del repositorio.

Figura 23

Historial del repositorio con el primer commit del proyecto.



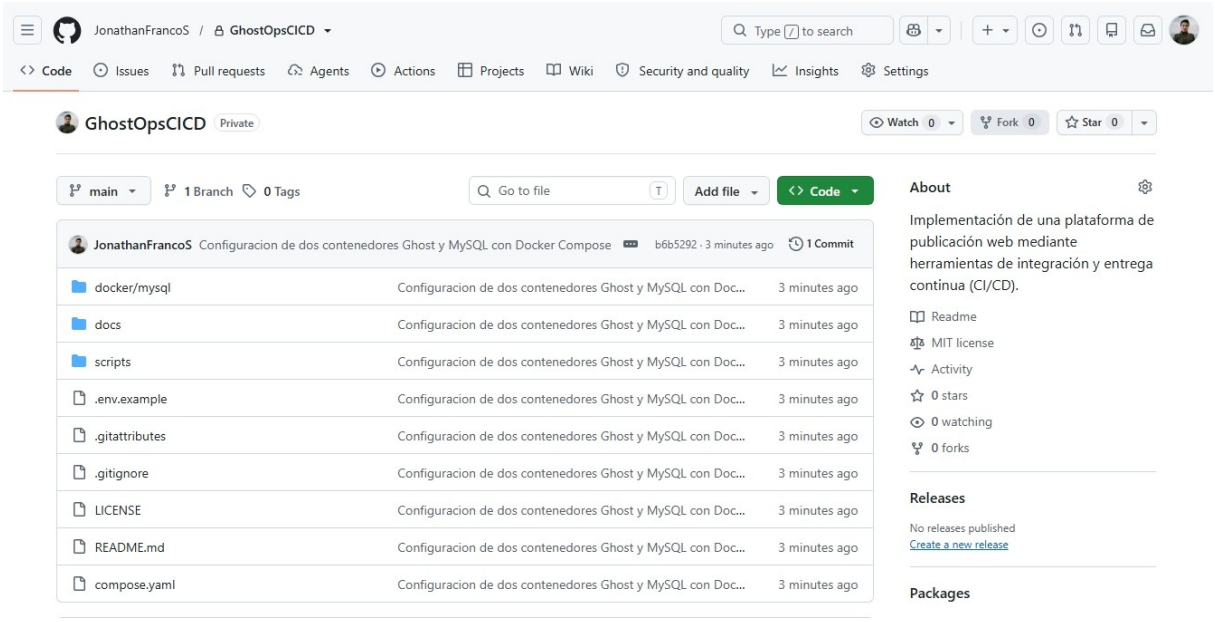
Nota. Captura de pantalla propia tomada durante el montaje del entorno el 6 de noviembre de 2026.

El historial registra el commit b6b5292, que incorpora la infraestructura inicial del proyecto. El detalle del archivo .env.example muestra que las variables se documentan con valores de ejemplo que deben sustituirse, y no con contraseñas reales.

Repositorio publicado en GitHub

Figura 24

Repositorio del proyecto publicado en GitHub.



Nota. Captura del repositorio del proyecto en GitHub, tomada el 7 de noviembre de 2026.

El repositorio quedó disponible en GitHub con la totalidad de los archivos del proyecto, su descripción, su licencia y su documentación. Esta publicación cumple el requisito de la entrega relativo al alojamiento del proyecto en una plataforma de control de versiones.

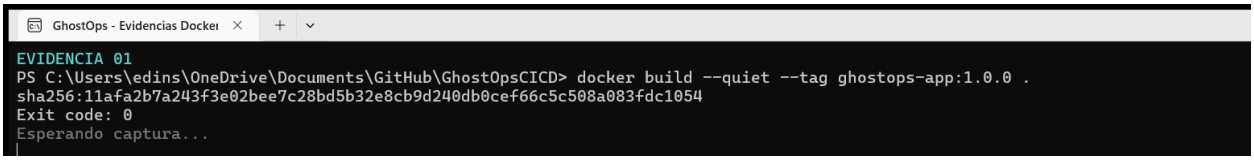
Imagen Propia E Integración Continua

Comprobado el funcionamiento de la infraestructura inicial, el equipo incorporó tres mejoras: la construcción de una imagen propia de la aplicación desde el repositorio, la fijación de las versiones exactas de las imágenes y la automatización de la validación del entorno mediante GitHub Actions. Las evidencias de esta segunda etapa se presentan a continuación.

Construcción de la imagen de la aplicación

Figura 25

Construcción de la imagen propia de la aplicación.



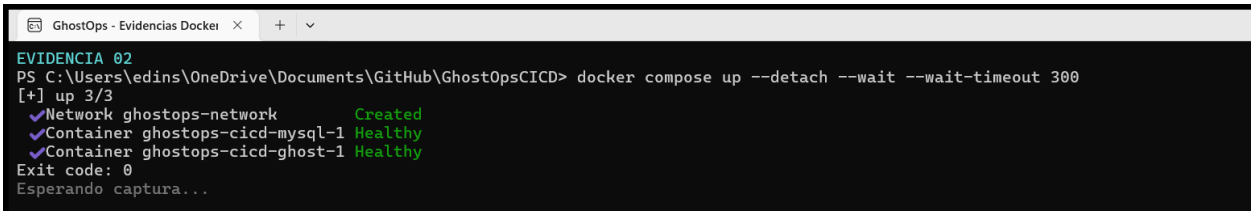
Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

Se incorporó un archivo Dockerfile que parte de la imagen oficial ghost:6.0.0-alpine y declara de forma explícita el directorio de trabajo, el puerto expuesto y el comando de inicio. La construcción generó la imagen ghostops-app:1.0.0 y finalizó con código de salida cero. De este modo la aplicación pasa a ejecutarse desde una imagen versionada por el propio proyecto, que el flujo de integración continua puede reconstruir y verificar en cada cambio.

Arranque con comprobaciones de salud

Figura 26

Inicio de los servicios esperando sus comprobaciones de salud.



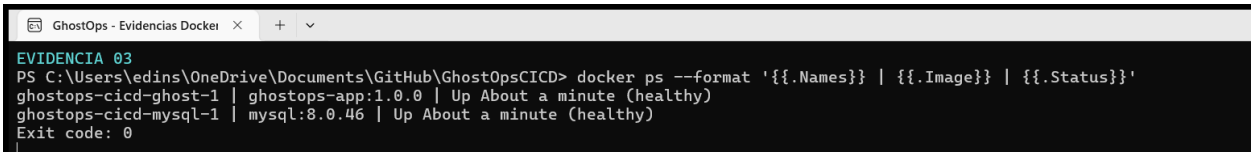
Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

Se añadió también una comprobación de salud a la aplicación, de modo que la orden docker compose up --wait no devuelve el control hasta que los dos servicios informan del estado

Healthy. Esta garantía es la que permite encadenar pruebas automatizadas con la certeza de que el entorno está listo y no solamente iniciado.

Figura 27

Estado de los contenedores con las versiones fijadas.

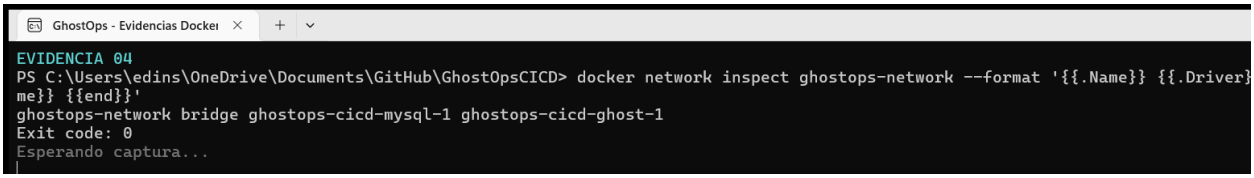


Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

La consulta del estado confirma que la aplicación se ejecuta ahora desde la imagen propia ghostops-app:1.0.0 y la base de datos desde mysql:8.0.46. Fijar las versiones exactas, en lugar de emplear la etiqueta latest, garantiza que cualquier integrante del equipo y el servidor de integración continua obtengan exactamente las mismas versiones utilizadas en estas pruebas.

Figura 28

Inspección de la red del proyecto tras la actualización.



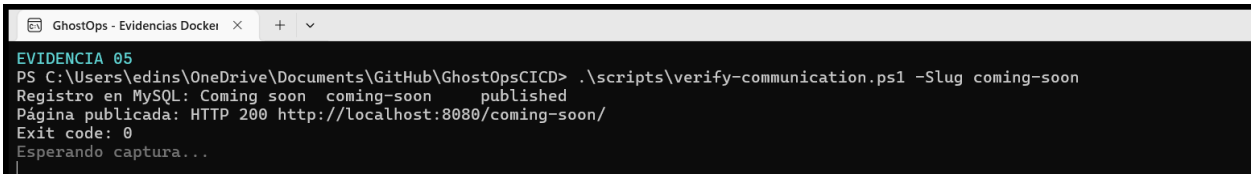
Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

La red ghostops-network mantiene su controlador bridge y conserva conectados a los dos contenedores del proyecto, de modo que el mecanismo de resolución de nombres sigue operando igual que en la etapa anterior.

Prueba de comunicación con el verificador

Figura 29

Verificación automatizada de la comunicación entre la aplicación y la base de datos.

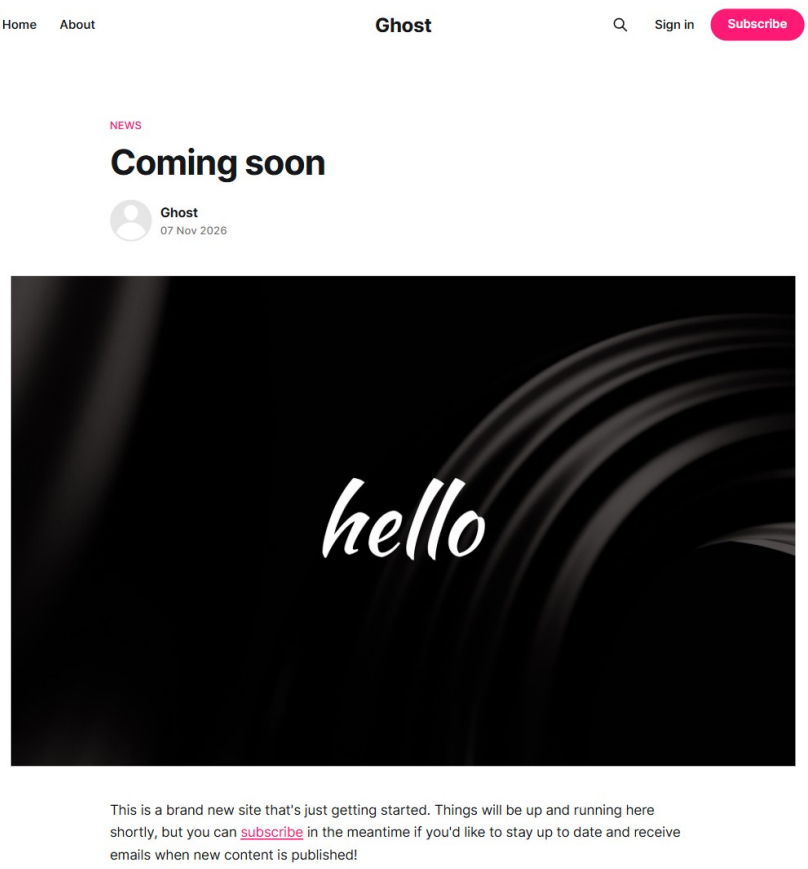


Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

Se incorporó al repositorio un script de verificación que realiza dos comprobaciones encadenadas. En primer lugar consulta directamente el contenedor de MySQL y recupera el registro de una publicación con su estado published. En segundo lugar solicita esa misma dirección a la aplicación y obtiene una respuesta HTTP 200. Dado que el registro consultado en la base de datos fue creado por Ghost desde el otro contenedor, esta prueba demuestra el flujo completo de escritura y lectura entre ambos servicios.

Figura 30

Publicación servida por la aplicación en el navegador.



Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

La misma publicación se visualiza desde el navegador en la dirección publicada por el contenedor de la aplicación, lo que cierra el recorrido completo del dato: se escribe desde la aplicación, se almacena en la base de datos y se sirve de regreso al usuario.

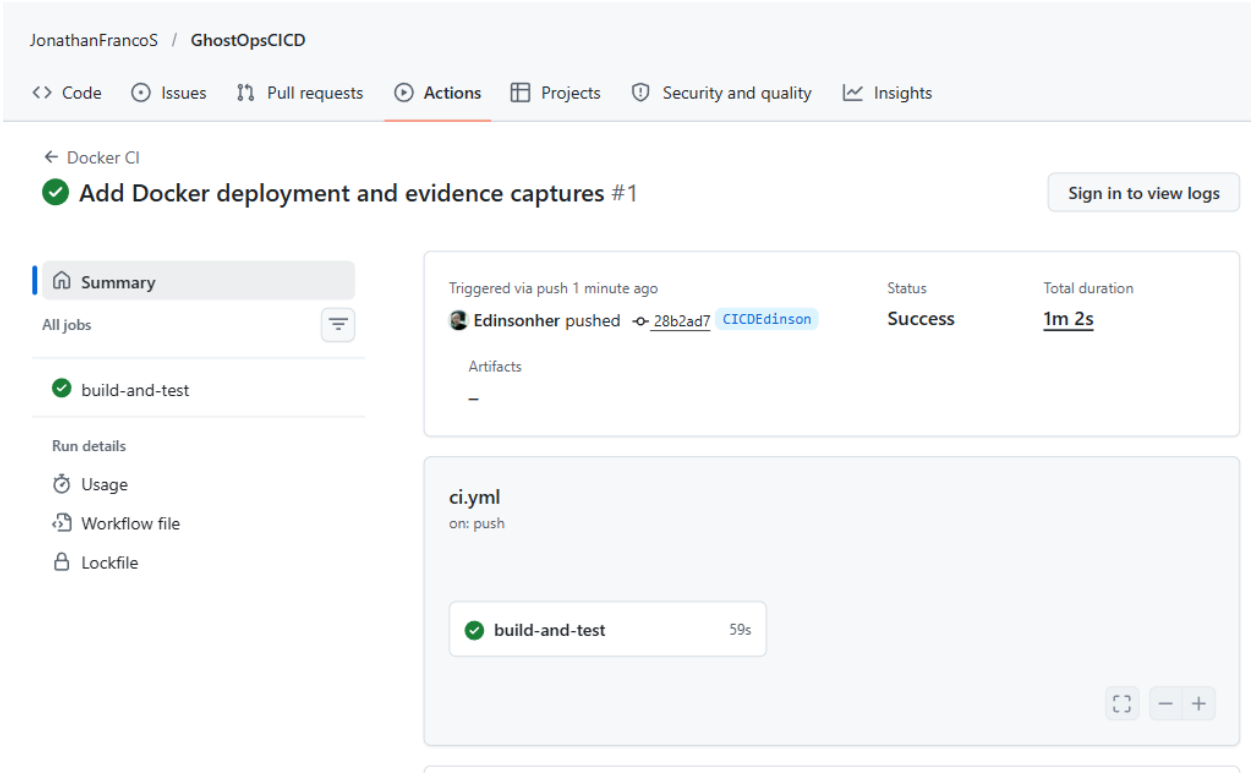
Flujo de integración continua

El repositorio incorpora un flujo de trabajo de GitHub Actions que se ejecuta automáticamente con cada envío de cambios y con cada solicitud de incorporación. El flujo valida la configuración de Docker Compose, construye la imagen de la aplicación, inicia ambos

servicios esperando sus comprobaciones de salud, verifica que la aplicación responda por HTTP y comprueba que Ghost haya inicializado la tabla posts dentro de MySQL.

Figura 31

Ejecución exitosa del flujo de integración continua.

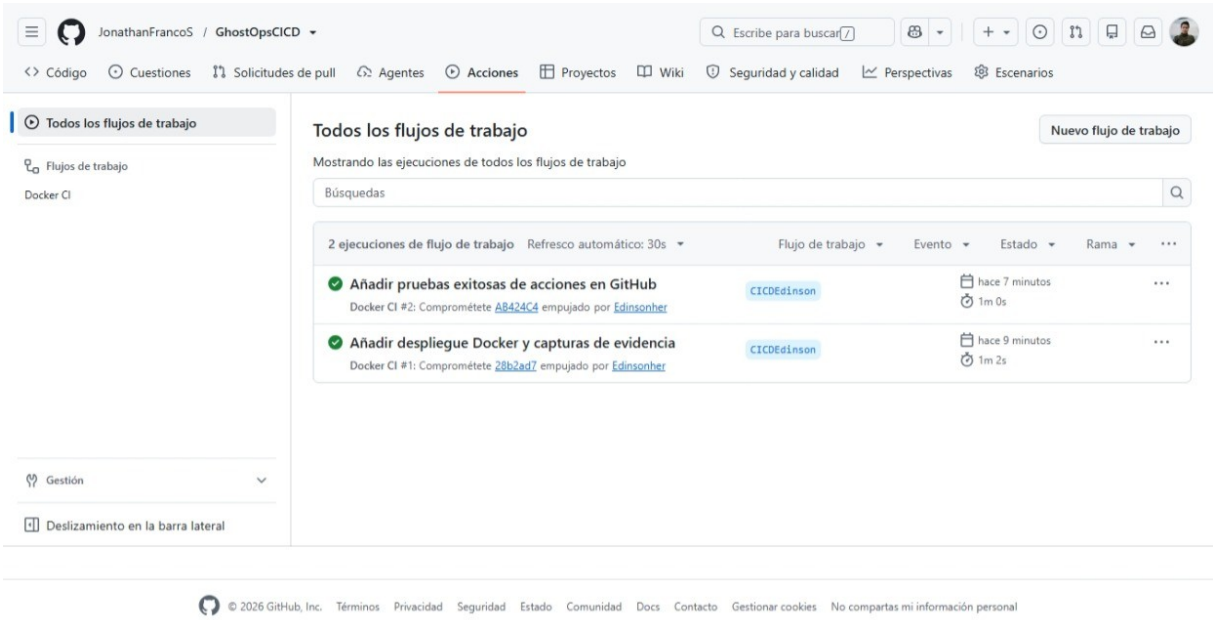


Nota. Captura de la ejecución del flujo Docker CI en el repositorio del proyecto, finalizada con estado Success. Recuperado de https://github.com/JonathanFrancoS/GhostOpsCICD/actions/runs/42404274686

La ejecución finalizó con estado Success en una duración de un minuto y dos segundos. Los avisos que acompañan al resultado corresponden a notificaciones informativas de mantenimiento de la plataforma y no afectan al resultado de la validación.

Figura 32

Historial de ejecuciones del flujo de integración continua.



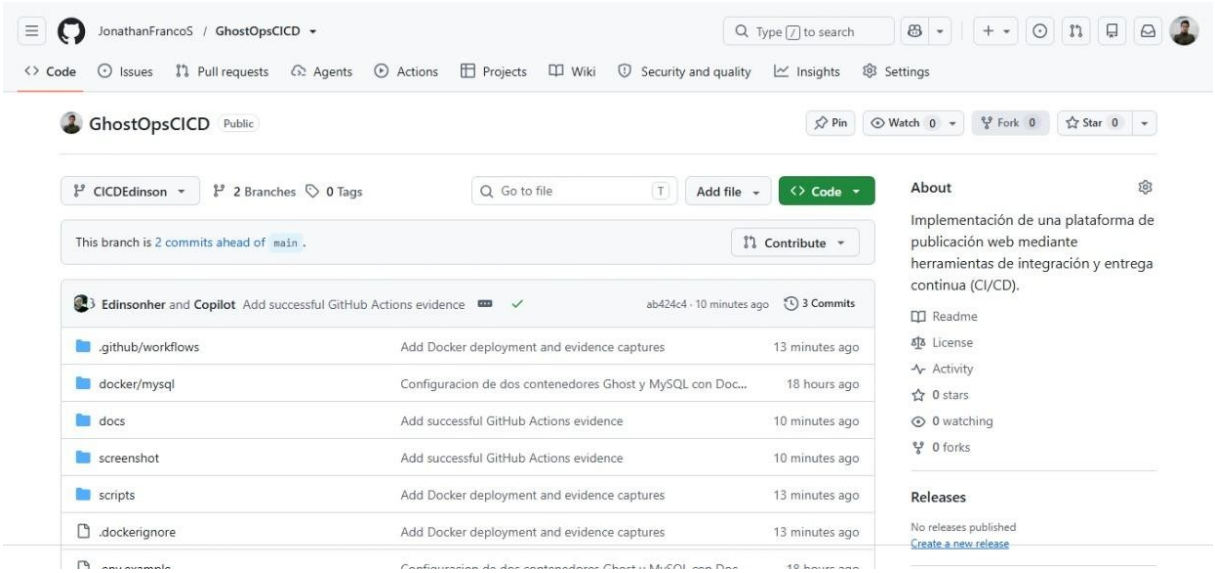
Nota. Captura del repositorio del proyecto en GitHub, tomada el 7 de noviembre de 2026.

El historial del flujo muestra las ejecuciones realizadas sobre la rama de trabajo, todas ellas finalizadas correctamente. Esta automatización constituye el primer paso del pipeline previsto para las entregas siguientes, ya que garantiza que ningún cambio pueda incorporarse al proyecto sin que el entorno completo haya sido construido y verificado.

Actualización de las ramas de trabajo

Figura 33

Rama de trabajo actualizada con los cambios de la segunda etapa.



Nota. Captura del repositorio del proyecto en GitHub, tomada el 7 de noviembre de 2026.

Los cambios de esta segunda etapa se incorporaron en una rama propia, que en el momento de la captura se encontraba dos commits por delante de la rama principal e incluía el archivo Dockerfile, el flujo de integración continua y la carpeta de capturas. El equipo acordó conservar la rama principal sin modificaciones durante la entrega, de manera que el historial de commits y el avance de cada integrante queden visibles como evidencia del trabajo realizado.

Conclusiones

La implementación permitió establecer una primera infraestructura basada en contenedores, separando la aplicación web de su sistema gestor de bases de datos. Se cumplió el objetivo de la entrega, puesto que el proyecto se encuentra alojado en un repositorio de GitHub y se construyeron dos contenedores Docker que se comunican entre sí, lo cual quedó demostrado mediante evidencias de ejecución real y no únicamente a través de la configuración declarada.

La comunicación entre contenedores no depende de direcciones IP sino del nombre del servicio, gracias a la resolución de nombres que Docker habilita en las redes definidas por el usuario. Este mecanismo es precisamente lo que hace que el entorno sea reproducible en cualquier máquina, condición indispensable para los procesos de integración continua.

Se comprobó igualmente que no basta con que un contenedor se encuentre iniciado para que su servicio esté disponible. La combinación de la comprobación de salud con la condición depends_on resultó indispensable para que el arranque fuera fiable, requisito necesario para que un flujo automatizado no falle de forma intermitente.

El paso de las imágenes oficiales a una imagen propia construida desde el repositorio supuso una mejora significativa, ya que traslada la definición de la aplicación al control de versiones y permite que cada cambio sea construido y verificado de forma automática antes de incorporarse al proyecto.

Finalmente, esta arquitectura constituye la base sobre la cual se incorporarán las herramientas de integración y entrega continua previstas para las entregas siguientes, dado que la contenerización elimina la diferencia entre el entorno de desarrollo y el del servidor de integración (Humble & Farley, 2010).

Enlace A Repositorio De GitHub

El código fuente completo del proyecto, junto con el archivo de orquestación, el flujo de integración continua, la documentación y las evidencias, se encuentra versionado y disponible en la siguiente dirección:

https://github.com/JonathanFrancoS/GhostOpsCICD

El equipo trabaja mediante ramas individuales, de modo que cada contribución queda registrada de forma independiente y la rama principal se conserva como referencia histórica de la evolución del proyecto.

Referencias

Docker Inc. (2026). *Docker documentation*. https://docs.docker.com/

Fowler, M. (2006). *Continuous integration*. https://martinfowler.com/articles/continuousIntegration.html

Ghost Foundation. (2026). *Ghost documentation*. https://ghost.org/docs/

GitHub Inc. (2026). *GitHub Actions documentation*. https://docs.github.com/actions

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Oracle Corporation. (2026). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

Politécnico Grancolombiano. (2026). *Proyecto de software basado en herramientas de integración continua* [Guía del proyecto grupal]. Módulo Énfasis Profesional I.